

Capstone Final Submission

Backend AI Engineering - Your 10x Solution

Book Insights API

Generated: 2026-09-06

Submit

- GitHub repository link
- Overview document: capstone/My 10x Solution - Harshini Priya.md

GitHub

```
origin https://github.com/Harshini-priya989/FlyRank_W1_A3_Containerized_Postgres (fetch)
origin https://github.com/Harshini-priya989/FlyRank_W1_A3_Containerized_Postgres (push)
```

Concepts

- API endpoints: FastAPI routes for books, search, recommendations, login, tagging, jobs, and reports.
- Database: SQLite persists users, books, jobs, reports, and usage/cost logs.
- Authentication: bearer tokens protect job, AI, report, and cost-log routes.
- Background jobs: import route uses FastAPI BackgroundTasks.
- Reporting: report route generates a PDF.
- Caching logic: list/search/recommendation endpoints use a 60-second cache.
- LLM integration: a narrow validated tagging/summarization endpoint logs estimated cost.

No swaps were needed.

Run

```
cd capstone
python -m pip install -r requirements.txt
python -m uvicorn src.app:app --reload --port 8010
```

Open <http://localhost:8010/docs> and use demo@example.com / password123 for login.

Recent Commits

```
7e022a5 Capstone: add Book Insights API
c93d360 W5 Stage 6: publish scraper evidence
d35c815 W5 Stage 5: survive failures and report run
75f0e6d W5 Stage 4: validate normalized records
1fc1fec W5 Stage 3: extract book details
c02aca6 W5 Stage 2: discover three catalogue pages
522bcbb W5 Stage 1: fetch and cache HTML
2d55d49 W5 Stage 0: classify scraping target
8564c18 Add Supabase auth assignment
805ede5 Add final database evidence
```

Overview

My 10x Solution - Harshini Priya

Problem

Small teams can collect useful public catalogue data, but the data often ends as a static JSON file that is hard to search, explain, or share. A person still has to open the file, scan records, compare prices, guess categories, and manually prepare a report. That makes a simple research task slower than it needs to be.

The users for this solution are interns, researchers, and small content teams who need a fast way to explore a collected dataset and turn it into a short report.

My 10x claim: finding relevant affordable books and producing a short report goes from about 20 minutes of manual scanning to under 2 minutes through searchable endpoints, cached recommendations, AI-style tagging, and one PDF report endpoint.

Solution

I built **Book Insights API**, a FastAPI backend that imports the validated W5 Books to Scrape dataset into SQLite, exposes search and recommendation endpoints, protects report/job/AI routes with bearer-token login, and generates a PDF report from the stored data.

The app starts from the W5 output file `scraper/output/books.json`. On startup, it initializes a SQLite database at `capstone/data/book\_insights.db`, creates a demo user, imports the 60 book records, classifies each book into a small category, and stores a short summary beside the original scraped facts.

Concepts Implemented

Concept	Implementation
API endpoints	FastAPI routes in `capstone/src/app.py` for health, login, books, search, recommendations, tagging, jobs, reports, and cost logs
Database	SQLite persists users, books, import jobs, generated reports, and usage/cost logs
Authentication	`/auth/login` returns a bearer token; protected routes require `Authorization: Bearer <token>`
Background jobs or cron jobs	`/jobs/import-books` queues the import using FastAPI `BackgroundTasks` so slow work can happen off the request path
Reporting - PDF	`/reports/books` generates a PDF report and `/reports/latest` downloads it
Caching logic	Search, list, and recommendation results are cached for 60 seconds
LLM integration	`/ai/tag` is one narrow validated AI-style job for category and summary generation, with a cost log entry

No swaps were needed. The project implements more than five concepts from the main concept table.

Non-Goal

I did not build a polished frontend or production-grade user account system. The capstone focuses on a clear backend workflow that can be run and reviewed quickly.

How To Run

```
```powershell
cd capstone
```

```
python -m pip install -r requirements.txt
python -m uvicorn src.app:app --reload --port 8010
```
```

Open Swagger:

```
```text
http://localhost:8010/docs
```
```

Demo login:

```
```json
{
 "email": "demo@example.com",
 "password": "password123"
}
```
```

Demo path:

1. Run `GET /health`.
2. Run `GET /books/search?q=history`.
3. Run `POST /auth/login` and copy the access token.
4. Click Swagger **Authorize** and paste `Bearer <access\_token>`.
5. Run `POST /ai/tag`.
6. Run `POST /reports/books`.
7. Run `GET /reports/latest` to download the PDF report.